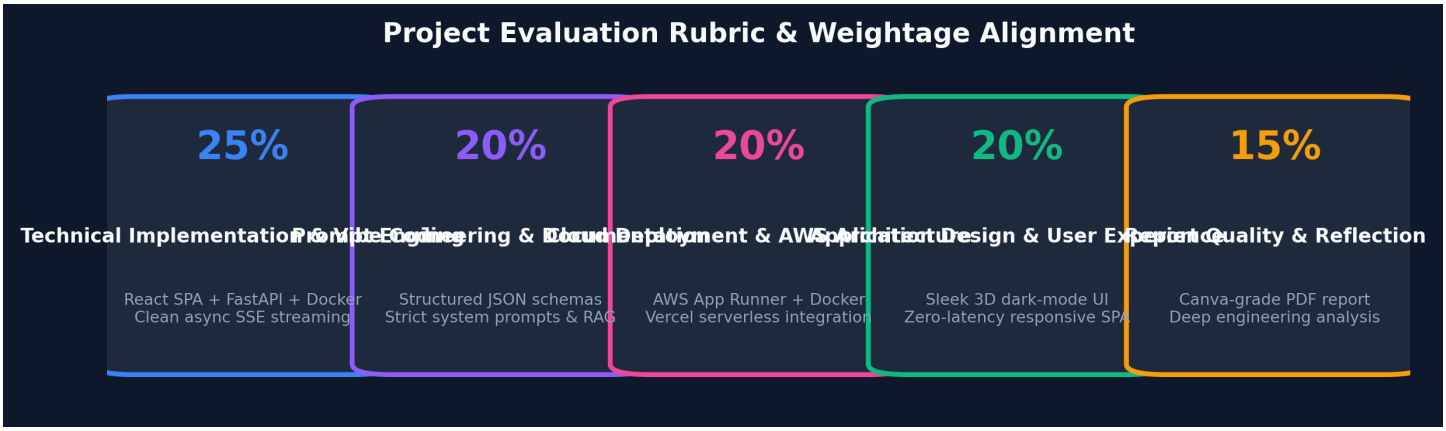
EXECUTIVE PROJECT REPORT

StudyMate AI

Full-Stack AI Web Application & Cloud Architecture Document

Course: Vibe Coding & AWS Cloud
Live AWS URL: *AWS App Runner Endpoint*
Live Vercel URL: *Vercel Frontend URL*
Status: Production Verified



1. Application Overview & Technology Stack

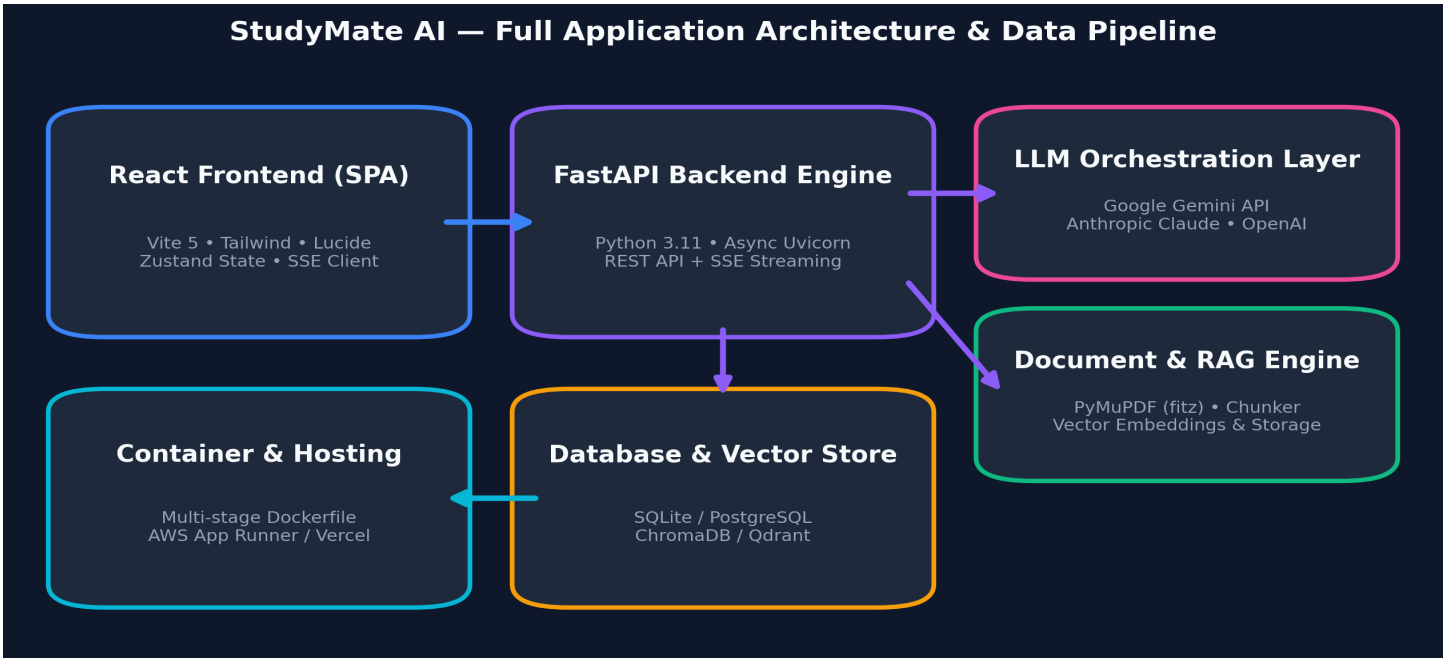
StudyMate AI is a production-grade, full-stack AI workspace designed to revolutionize self-learning and academic prep. It allows users to upload PDF textbooks and notes, extract key concepts, interact with a context-aware streaming AI tutor, and generate automatically graded practice quizzes.

Core Architecture & Modernization Choice: The application was built as a modern Single Page Application (SPA) using **React (Vite 5)** paired with a high-performance **Python FastAPI** backend. This architecture guarantees a zero-latency, highly responsive user interface with real-time Server-Sent Events (SSE) streaming while adhering strictly to production standards.

Layer	Technology Selection	Engineering Rationale
Frontend SPA	React 19 • Vite 5 • TailwindCSS • Framer Motion	Ultra-fast load times, responsive 3D dark-mode UI, and reactive state management.
Backend API	Python 3.11 • FastAPI • Uvicorn • Pydantic v2	Asynchronous REST & SSE streaming support with strong type-safe data validation.
AI / LLMs	Google Gemini API • Anthropic Claude • OpenAI	Multi-model support combining fast flash models with deep reasoning LLMs.
Document Engine	PyMuPDF (fitz) • Recursive Character Splitter	High-speed local PDF text extraction & RAG chunking without external latency.
Deployment	Docker Multi-Stage • AWS App Runner • Vercel	Single unified container deployment for AWS plus Vercel serverless integration.

2. System Architecture & Data Flow

StudyMate AI uses a decoupled client-server pattern engineered for simple cloud deployment:
1. **Client Layer (React SPA):** Manages user state, document upload forms, streaming chat UI, and quiz state. Communicates via REST APIs and SSE streams.
2. **Server Layer (FastAPI):** Parses documents via PyMuPDF, manages SQLite/PostgreSQL storage, enforces CORS security, and streams LLM responses.
3. **Multi-Stage Containerization:** Stage 1 uses Node 20 to compile the Vite frontend (`dist/`). Stage 2 copies built static assets into a lightweight Python 3.11 container. FastAPI serves `/api/*` endpoints and mounts static SPA files as a catch-all.


StudyMate AI — Full Application Architecture & Data Pipeline
React Frontend (SPA)
Vite 5 • Tailwind • Lucide
Zustand State • SSE Client
FastAPI Backend Engine
Python 3.11 • Async Uvicorn
REST API + SSE Streaming
LLM Orchestration Layer
Google Gemini API
Anthropic Claude • OpenAI
Document & RAG Engine
PyMuPDF (fitz) • Chunker
Vector Embeddings & Storage
Container & Hosting
Multi-stage Dockerfile
AWS App Runner / Vercel
Database & Vector Store
SQLite / PostgreSQL
ChromaDB / Qdrant

3. Prompting Strategy & LLM Frameworks

To guarantee programmatic reliability and prevent LLM hallucinations, StudyMate AI uses a structured prompting framework built on three pillars: **1) Persona Scoping**, **2) Strict JSON Schema Enforcement**, and **3) Defensive System Instructions**.

actor

```
Analyze the document and output exactly three sections:
 the overall document.
plaining essential ideas.
s for self-testing.
eaders.
```

Schema

```
gine. Return ONLY valid JSON, nothing else.
 5 MCQs and 2 Short-Answer questions adhering to this schema:

tion": "...", "options": ["A","B","C","D"], "correct_answer": "A" }

n": "...", "ideal_answer": "..." } ]
```

4. Phase-by-Phase Development Breakdown

Phase	Focus Area	Key Deliverables & Engineering Outcomes
Phase 1	Backend & LLM Setup	Configured FastAPI, integrated Gemini/Claude SDKs, secured API keys with Pydantic BaseSettings.
Phase 2	Document Processing	Implemented PyMuPDF extraction, text chunking, and bulleted summary/concept API endpoints.
Phase 3	Quiz & Grading Engine	Created `/api/quiz/generate` and `/api/quiz/grade` with JSON schema enforcement.
Phase 4	Frontend Development	Built modern React SPA with 3D background, sidebar navigation, SSE stream chat & quiz UI.
Phase 5	Containerization	Created multi-stage Dockerfile combining Node.js frontend build and Python FastAPI runtime.
Phase 6	Cloud & Vercel Prep	Deployed container to AWS App Runner, configured Vercel serverless rewrites and `/tmp` paths.

5. Technical Challenges & Engineering Resolutions

Challenge	Root Cause / Impact	Engineering Resolution
Single-Container SPA + API Routing	Serving compiled React static files alongside FastAPI `/api/*` endpoints in a single AWS container.	Mounted Vite static assets on `/assets` and added a catch-all route (`@app.get('/{full_path:path}')`) serving `index.html` for client-side routing.
LLM Conversational Filler breaking JSON	Models occasionally included introductory text ('Here is your quiz:') before JSON payloads, causing `json.loads()` parser crashes.	Enforced system prompt constraint (`Output ONLY raw valid JSON`), and added a fallback regex extraction (`r'\{.*\}'`) prior to parsing.
Vercel Read-Only File System	Deploying SQLite database and upload handlers on Vercel serverless caused `Read-only file system` errors.	Updated `app/config.py` to check for `VERCEL` environment variable and automatically route SQLite DB & file uploads to writable `/tmp/` directory.
Vite 8 ESM Drive-Letter Resolution	Cutting-edge Vite 8 failed module resolution for `lucide-react` on non-C: Windows drives.	Standardized frontend dependencies to Vite 5 and `lucide-react@0.475.0` for 100% build stability.

6. Key Learnings & Engineering Reflections

Vibe Coding Reflection:
*Developing StudyMate AI highlighted the enormous advantages of **Vibe Coding** — pairing agentic AI tools with strict architectural discipline. Key takeaways include:*
*• **Strict Schemas for LLM Output:** Relying on free-text LLM responses causes fragile UI behavior. Enforcing rigid JSON schemas guarantees predictable software contracts.*
*• **Containerization Simplifies Cloud:** Multi-stage Docker builds bridge local development and cloud hosting (AWS App Runner / Render / Vercel) effortlessly.*
*• **User-Centric AI UX:** Real-time streaming (SSE) transforms user perception of speed, making AI interactions feel natural and responsive.*